

Spotify Data Studio

song.py:

در ابتدا به سراغ طراحی ماثول song که برای کپسوله سازی اطلاعات مربوط به آهنگ ها و افزودن آهنگ های جدید بود، رفتم. در این ماثول یک کلاس song تعریف کردم و هر کدام از ستون های دیتاست را به عنوان یک Attribute برای آبجکت های این کلاس در نظر گرفتم و تعریف کردم. برای هر کدام از این اتریبیوت ها یک property setter مناسب طراحی کردم تا برای افزودن آهنگ های جدید از طریق CLI دچار مشکل نشویم. در نهایت در این کلاس باید متدی طراحی میکردم تا بتوانیم با استفاده از آن اطلاعات ترک های جدید را از ورودی بخوانیم که در این بخش چندین ایده ی مختلف را از جمله استفاده از یک classmethod امتحان کردم اما در نهایت برای ایجاد یک Interactive input از staticmethod استفاده کردم.

- [GeeksforGeeks1](#)
- [GeeksforGeeks2](#)
- [GeeksforGeeks3](#)

data_loader.py:

ماثول بعدی که به سراغ طراحی آن رفتم، ماثول data_loader بود که باید فایل دیتاست را لود میکرد و هر سطر از آن را تبدیل به یک آبجکت از کلاس song میکرد. در این ماثول سه کلاس DataLoader ، ObjectCreator و Reporter را طراحی کردم تا هم به وظایف خود عمل کنند هم از SRP پیروی کرده باشم و هر کدام از کلاس ها به یک وظیفه عمل کنند و تنها یک دلیل برای تغییر داشته باشند.

- [GeeksforGeeks](#)

data_finder.py:

در این ماثول کلاس DataFinder را تعریف کردم تا برای ایجاد فایل نصبی و setup برنامه بتوانم فایل dataset.csv را بدون ایجاد مشکل پیدا کنم و هرکجا نیاز به این فایل باشد، برای بروزرسانی آن یا برای خواندن اطلاعات از روی آن، از متد های همین ماثول استفاده کردم تا از اصل DRY پیروی شده باشد.

data_cleaner.py:

این ماثول یکی از چالش برانگیزترین ماثول های این برنامه بود؛ در این ماثول دیتا های گمشده و دیتا های پرت تبدیل به دیتا های منطقی شده و یا حذف میشوند. در این ماثول یک کلاس PreProcessor نوشتم تا آن

دسته از دیتا های گمشده که عددی نیستند و نمیتوان به مقدار حقیقی آنها دست یافت را حذف کند و دیتا های تکراری را نیز از دیتاست حذف کند.

برای دیتا های گمشده 3 روش برای جایگزینی طراحی کردم؛ میانه، میانگین و KNN. برای پیاده سازی هرکدام از این روش ها نیاز به تسلط به سینتکس کتابخانه Pandas بود و البته برای KNN علاوه بر Pandas باید با کتابخانه sklearn نیز کار میکردم. جهت پیدا کردن ستون های عددی و پر کردن خانه های NaN آن ستون ها با روش مورد نظر به چند سایت رجوع کردم تا متوجه سینتکس درست و مورد نیاز این بخش بشوم.

برای دیتا های پرت نیز از سه معیار و روش استفاده کردم؛ اولین معیار مورد استفاده در این برنامه، IQR بود که به صورت دستی چارک ها را محاسبه کردم و کران بالا و پایین را تعیین کردم و با ساختن یک Boolean series دیتاهایی که پرت بودند را به NaN تبدیل کردم تا با استفاده از یکی از روش های از پیش تعریف شده دوباره یک مقدار برای آنها تعیین کنم. معیار دوم شاخص ZScore بود که ابتدا با چند سرچ ساده با نحوه محاسبه و منطق آن آشنا شدم و در نهایت آن را با استفاده از کتابخانهی scipy پیاده سازی کردم و دوباره با استفاده از یک سری بولین دیتاهای پرت را تبدیل به NaN کردم. شاخص سوم هم Winsorization بود که در آن یک کران پایین و یک کران بالا برای ستون تعیین کردم و دیتا هایی که کمتر از کران پایین بودند را تبدیل به کران پایین و دیتا های بیشتر از کران بالا را به کران بالا تبدیل کردم. برای پیاده سازی این بخش نیز از سینتکس های Pandas استفاده کردم.

برای آشنایی بیشتر با نحوه محاسبه این شاخص ها و نحوه پیاده سازی آنها در پایتون و سینتکس های مربوط به آنها نیز به چندین سایت مراجعه کردم که در ادامه ذکر شده اند.

در طراحی این ماژول سعی کردم که طبق اصل DIP فارغ از نحوه جایگزینی داده های گمشده و یا هندل کردن داده های پرت، یک رابط کلی طراحی کنم و براساس این رابط کلی در فایل main توابع را فراخوانی و استفاده کنم.

- [GeeksforGeeks1](#)
- [GeeksforGeeks2](#)
- [GeeksforGeeks3](#)
- [GeeksforGeeks4](#)
- [ZScore](#)
- [IQR](#)

data_analyzer.py:

در این ماژول با استفاده از کتابخانه Pandas تحلیل های متنوعی ارائه شده. عده ای از این تحلیل ها و آمار ها به طور کلی و عده ای دیگر بر اساس ژانر مورد انتخاب کاربر هستند. تنها چالش این ماژول استفاده از سینتکس های کتابخانه Pandas بود. در این ماژول با استفاده از یک کلاس DataAnalyzer و متد های گوناگون تقریباً از تمامی ستون های دیتاست آمار و تحلیل ارائه شده است.

- [GeeksforGeeks1](#)
- [GeeksforGeeks2](#)

data_writer.py:

این ماژول در پروژه نقش نوشتن اطلاعات جدید بر دیتاست و بروزرسانی آن را بر عهده دارد. در این ماژول از از کلاس DataFinder در ماژول DataLoader استفاده شده تا فایل دیتاست را همواره پیدا کند و اطلاعات جدید را روی آن بنویسد و آن را آپدیت کند.

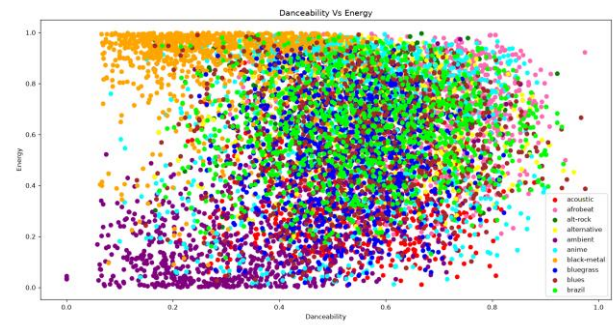
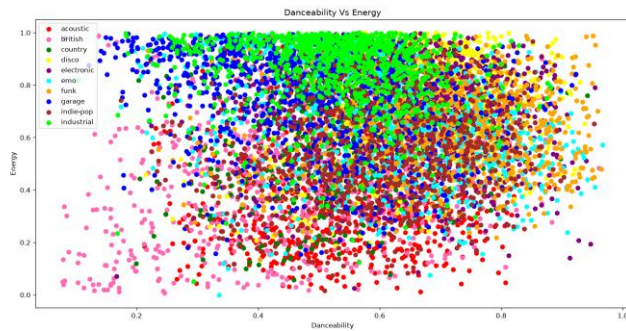
- [StackOverflow](#)

data_visualizer.py:

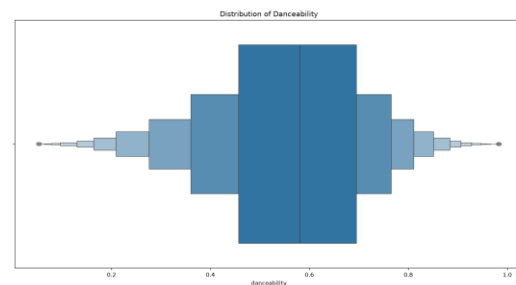
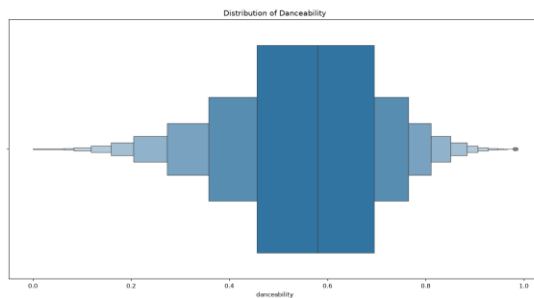
این ماژول قرار است با نمودار های متنوع دید بازتری از دیتا ها به کاربر بدهد. در این ماژول استفاده ی زیادی از کتابخانه ی matplotlib برای رسم نمودار ها شده است. همچنین برای رسم نمودار همبستگی Heatmap از ماژول data_analyzer جهت دستیابی به Correlation Matrix استفاده کردم. برای رسم Box plot ها هم از کتابخانه seaborn استفاده کردم. همچنین یک Bar plot و یک Pie Chart برای ژانرها اضافه کردم که 10 ژانر اولی که بیشترین ترک را دارد نشان میدهد.

برای استفاده از matplotlib داکيومنت کامل این کتابخانه را مطالعه کردم و برای استفاده از ماژول seaborn هم سرچ کردم تا سینتکس مناسب را پیدا کردم.

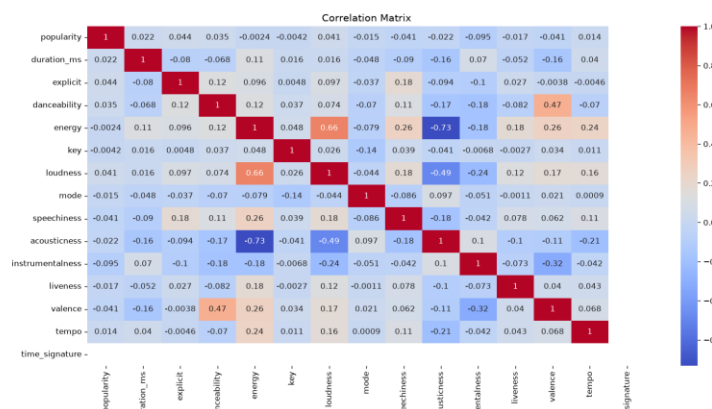
- [W3Schools1](#)
- [W3Schools2](#)
- [W3Schools3](#)
- [W3Schools4](#)
- [GeeksforGeeks1](#)
- [GeeksforGeeks2](#)



نمودار های Scatter plot قبل و بعد از پاکسازی دیتا با zscore و median



نمودار های Box plot قبل و بعد از پاکسازی دیتا با zscore و median



نمودار Heatmap بعد از پاکسازی دیتا با zscore و median

بزرگترین چالش های این پروژه بخش پاکسازی دیتا ها و مازول data_cleaner بود که تک تک کلاس های مورد نیاز در این بخش را با جستجو در منابع موجود در اینترنت طراحی کردم و نوشتم.

هر کلاس در این پروژه تنها یک دلیل برای تغییر دارد (طبق اصل SPR) و در کلاس هایی که ارث بری صورت گرفته است به طور کامل polymorphism رعایت شده و هر کلاس تنها متد هایی را تعریف کرده که به آنها نیاز دارد (طبق اصل ISP) و وابستگی کلاس ها به رابط یا همان Abstract base class اصلی است و به جزئیات نحوه impute یا handle کردن بستگی ندارد (طبق اصل DIP).

در صورتی که بخواهیم نحوه جدیدی از آنالیز دیتاها اضافه کنیم و یا بخواهیم نوع جدیدی از نمودار را بکشیم و یا حتی اگر بخواهیم با روش جدیدی دیتا ها را پاکسازی کنیم، این کار بدون نیاز به تغییر باقی بخش های کد صورت خواهد گرفت (طبق اصل OCP).

در ساختار مازول های این برنامه اصول SOLID به طور کامل پیاده سازی شده و هیچ کجا کد یا منطقی خودش را تکرار نکرده که این نشان دهنده پیروی از اصل DRY است.

دیگر منابع این پروژه:

- [GeeksforGeeks](#)
- [YouTube](#)